新时代 AI Agent 安全：一个安全从业者近期的几点思考

最近这段时间，我花了些时间在看和写各种 Agent，和各家的安全实践。

看得越多，我越觉得一件事很明显：Agent 安全已经不能只按“大模型安全”的思路来理解了。

以前我们谈大模型安全，更多会盯着两个点。一个是 prompt injection，一个是输出内容是否合规。这个角度当然还重要，但如果放到今天很多真正能“干活”的 Agent 身上，就明显不够用了。因为它们已经不只是回答问题了，它们会查知识库，会调 API，会读网页，会写状态，甚至会真正去执行动作。

所以，作为一个安全从业者，我现在越来越倾向于把 Agent 看成一个任务执行系统，而不是一个会聊天的大模型外壳。这个视角一变，很多安全判断也会跟着变。

一、AI Agent 的基本组成

我觉得很多关于 Agent 安全的讨论，一上来就容易掉进一个误区，就是默认 Agent 只是“模型 + prompt + tool call”。

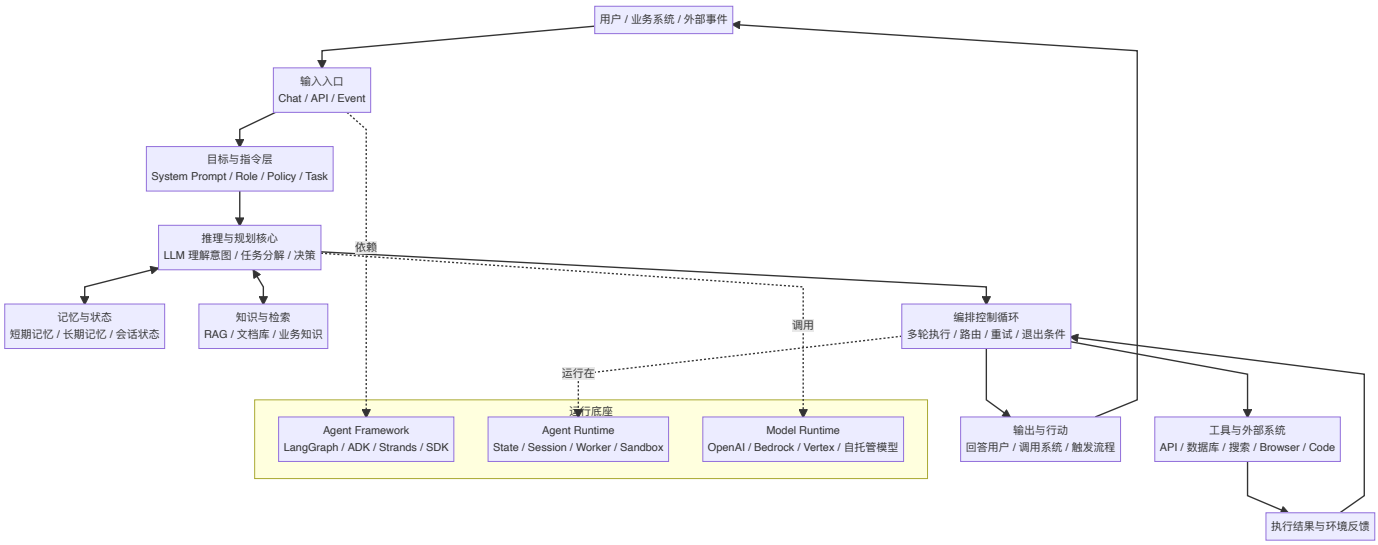
但如果你真的去拆一个能跑起来的 Agent，会发现事情远不止这么简单。它至少会包含下面这些部分：

组件	它在做什么	安全上容易出什么问题
输入入口	接收用户请求、业务事件、API 调用、多模态内容	恶意输入、伪造任务、跨渠道注入
目标与指令层	定义角色、目标、约束和系统规则	策略污染、越权指令、提示词被覆盖
推理与规划核心	理解目标、拆任务、决定下一步做什么	概率性误判、危险路径选择、错误放大
编排控制循环	把推理、工具调用、状态推进串成闭环	死循环、异常重试、错误扩散
记忆与状态	保存上下文、长期记忆、任务状态	记忆投毒、跨用户污染、越权复用
知识与检索	从知识库、网页、数据库里拿信息	检索投毒、敏感信息带出、结果误用
工具与外部系统	调 API、查库、发消息、跑浏览器、执行代码	权限扩张、工具滥用、供应链风险
输出与反馈	给用户结果，或把结果送回下一轮继续推理	误执行、误触发、错误反馈闭环
运行底座	框架、运行时、会话管理、沙箱、执行器	身份混淆、隔离失效、审计缺失

也就是说，Agent 从来都不是一个单点能力，它更像是一条会持续接收信息、持续做判断、持续调用外部能力的执行链。

二、AI Agent 的通用结构

下面这张图，是我这段时间看各类 Agent 之后，觉得比较通用的一种抽象。它不属于某一家厂商，但基本能覆盖今天大部分 Agent 系统的关键部件。



这张图真正提醒我的，不是“组件很多”这件事，而是 Agent 的很多环节都会继续反过来影响下一步决策。

输入会影响推理。

检索会影响推理。

工具返回结果会影响推理。

记忆会影响推理。

环境反馈还是会影响推理。

所以从安全角度看，Agent 不是一次请求处理完就结束的系统，而是一个不断吸收输入、不断改变状态的系统。

三、AI Agent 安全不只是提示词安全

如果只把 Agent 看成一个大模型，那安全工作就很容易收缩成两件事：防注入，管输出。

但问题在于，今天很多 Agent 已经不是“说完就结束”了，而是“说完之后还会继续做”。一旦进入这个阶段，风险就会明显变样。

现在更关注下面几类问题：

- 输入入口会不会被恶意任务或伪造指令污染。
- 指令层会不会被覆盖，导致原本的边界慢慢漂移。
- 检索、记忆和历史上下文会不会把错误内容持续带进系统。
- 工具调用会不会把一次错误判断放大成真实动作。
- 运行时和沙箱设计不当时，模型层问题会不会直接外溢到主机、网络和业务系统。

换句话说，**Agent 安全已经并不是单点的防护，而是一个覆盖整个运行链路的系统级问题。**

四、传统应用安全在 AI Agent 时代面临的挑战

在传统 Web 应用、移动应用或者 SaaS 系统里，很多安全设计默认成立的前提其实比较稳定，都是由一个确定的输入得到一个确定的输出：流程大体固定，调用边界相对清楚，用户自己显式触发敏感动作，输入点也比较集中，日志通常能覆盖主要问题。可一旦系统开始引入 Agent，这些前提就会一层层松动。

先看一个直观一点的对比：

对比维度	传统应用安全假设	Agent 系统中的变化
行为路径	流程固定、接口可枚举	路径动态、推理驱动、难以完全枚举
权限控制	用户直接操作，应用内校验为主	Agent 代执行，需要跨链路委托控制
调用对象	固定服务依赖	动态工具、MCP、外部系统持续扩展
输入风险	主要来自用户输入	用户输入、工具返回、记忆、检索内容共同构成上下文风险
结果风险	回答错误、内容违规、信息泄露	进一步扩展到真实动作执行和业务后果
运行环境	边界相对稳定	运行时更开放，包含浏览器、沙盒、代码执行等新环境
审计方式	接口日志和数据库日志即可覆盖大部分问题	需要覆盖推理、检索、工具调用、授权、执行与回放全链路

从这个对比里其实能看出一个很重要的点：传统安全体系并不是失效了，而是覆盖范围不够了。

身份认证、权限控制、网络隔离、日志审计、数据安全、漏洞管理，这些东西当然还都要做，而且一个都不能少。

问题在于，**Agent 改变了系统运行的基本形态**。传统应用里很多原本默认成立的安全假设，到了 Agent 环境下已经不再那么稳了。

1. **从确定性流程，变成概率性决策**

   传统应用的大部分执行路径是预定义的。用户点击哪个按钮，触发哪个接口，后端按什么规则处理，通常都比较确定，也比较容易枚举。

   但 Agent 不是这样。Agent 的核心行为来自模型推理，而由于现在llm的原理，导致推理本身就具有概率性。

   - 同一个目标，Agent 可能拆解成不同的任务路径。
   - 同一个问题，Agent 可能选择不同工具、不同顺序、不同参数。
   - 同一个上下文，在不同模型版本或不同提示条件下，决策结果也可能变化。

   这意味着安全问题不再只是“某个接口有没有权限校验”，还要回答“Agent 会不会在动态决策中走到一条原本没预想到的危险路径”，不能复用通过传统安全中的白名单思路来一劳永逸。

2. **从固定接口调用，变成动态工具编排**

   过去的应用依赖通常比较固定，前端调哪些后端接口，后端依赖哪些微服务，服务之间依赖关系相对清晰，边界大体都清楚。

   但在 Agent 系统中，工具调用往往是动态选择的。

   - Agent 会根据上下文自主决定调用哪个工具
   - 一个工具的结果又会触发下一轮工具选择
   - 工具集合可能持续扩展，包括内部 API、MCP server、数据库、SaaS、浏览器和代码执行环境

这使得攻击面从“接口安全”扩展为“工具链安全”。风险也从单次 API 越权，上升为整条执行链路中的权限扩张、供应链污染和间接注入。

3. **从用户直接操作，变成 Agent 代理执行。**

   传统应用中，大多数敏感操作都由用户显式触发，谁发起、谁确认、谁负责，关系相对清楚。

   - 用户自己发起查询
   - 用户自己点击审批
   - 用户自己提交删除

   因此传统安全设计主要围绕“用户是谁”，“用户能不能做这个动作”展开。

   到了 Agent 时代，情况发生了变化。用户越来越多地不是直接操作系统，而是把目标交给 Agent，由 Agent 代为规划和执行。

   - 用户说“帮我整理今天的待办并发给团队”
   - 用户说“查询客户信息并生成一份跟进建议”
   - 用户说“根据合同内容起草回复邮件并发送”

   这时，真正执行动作的是 Agent，而不是用户的显式点击。所以这里的安全设计就不能只停留在“用户有没有权限”，还得继续往下看：Agent 能不能只在用户授权范围内代表用户行动。

4. **从静态权限控制，变成委托链路控制**

   传统权限模型通常只需要处理一段较短的链路：

   `用户 -> 应用 -> 数据库`

   而在 Agent 系统中，链路会迅速拉长：

   `用户 -> Agent -> 编排层 -> 工具网关 -> 下游服务 -> 数据系统 -> 外部系统`

   这会带来几个新的安全难点：

   - 身份如何跨多个组件连续传递
   - 下游系统如何识别最终用户，而不是只看到 Agent 服务账号
   - Agent 是否会在调用链中获得比用户本身更高的权限
   - 多租户环境中，跨租户边界是否会在工具编排阶段被破坏

   也就是说，Agent 安全中的“权限”已经不只是一个应用内的访问控制问题，而是一个跨多层系统的委托控制问题。

5. **从数据输入风险，扩展为上下文污染风险**

   传统应用确实也面对输入风险，比如 SQL 注入、XSS、文件上传攻击，但其风险面主要集中在显式输入和明确的数据边界上。

   Agent 系统则不一样。它的输入范围远比传统应用更广：

   - 用户输入
   - 系统提示词
   - 工具描述
   - 工具返回内容
   - 检索文档
   - 长期记忆
   - 历史对话
   - 外部网页

这些内容都会进入 Agent 的推理上下文。一旦这里面任何一部分被污染，影响的就不只是某个字段或某次请求，而可能是整个决策链。所以这类问题更应该是“上下文污染”和“上下文治理”，而不只是传统意义上的输入校验。

6. **从边界清晰系统，变成开放耦合系统**

   传统企业应用通常边界相对稳定。

   - 业务系统之间的依赖关系清楚
   - 接口和角色模型相对固定
   - 环境通常是内聚的、可控的

   而 Agent 系统天然倾向于开放耦合：

   - 接入更多第三方工具和 SaaS
   - 引入 MCP、插件、远程浏览器、外部搜索
   - 通过知识库和网页检索动态吸收外部信息
   - 强烈的 Agent 自身的自我进化需求

   这使得安全模型必须从“保护一个封闭系统”，转向“管理一个持续变化的开放执行网络”。

7. **从结果输出安全，变成执行后果安全**

   传统大模型应用早期主要关注回答是否合规、是否有害、是否泄露隐私。这些问题当然仍然重要，但在 Agent 场景里，它们已经不是全部。

   因为 Agent 不只是“说”，它还会“做”。

   - 它会查数据库
   - 它会打开网页
   - 它会调用 API
   - 它会运行脚本
   - 它会发邮件、提工单、触发审批

   这意味着安全问题的重心会从“输出内容本身是否安全”，进一步转向“执行动作及其后果是否可控”。

   一句话概括就是：

   在传统应用里，系统主要承担的是信息处理风险；在 Agent 应用里，系统开始承担行动执行风险。

五、Agent 时代，安全重心正在发生变化

在 Agent 时代面临的这些关键的安全问题，在学习了很多大佬的思路后，目前针对Agent 安全时代下最重要的几个安全能力，提出一些自己的拙见

一、Agent 的身份验证与委托授权（Agent Identity）

在 AI 时代，Agent 场景下最容易发生的问题就是**混淆身份与委托越权**

而身份上最容易出现的问题其实不是“用户有没有登录”，而是“当前到底是谁在代表谁做事”。

传统应用里，用户通常直接操作系统，身份链路相对简单，应用只需要校验当前用户是否有权限访问某个接口、某份数据、某个按钮即可。

但在 Agent 场景里，情况发生了变化。用户越来越多是把目标交给 Agent，再由 Agent 去访问工具、获取数据、调用系统、执行动作。这个时候，系统需要校验的就不再只是“用户有没有权限”，而是整条链路上的身份和授权关系：

- 用户是否有权发起这个代理请求
- Agent 是否有权代表该用户去调用某个工具或服务
- 下游工具或服务是否能够识别 Agent 代表的是哪一个真实用户
- 工具返回的数据是否是该用户本来就有权访问的数据

也就是说，在 Agent 时代，身份验证必须从单点登录校验，升级为**端到端的全链路委托授权**。

而这一部分的安全建设至少要包括以下几个方面：

1. **Agent 入方向的认证和授权**

对要求进行代理的用户进行认证和授权，包括通过第三方身份提供商登录进来的用户。系统要明确知道是谁发起了请求、请求的目标是什么、是否允许 Agent 代为执行。

2. **外部工具或服务对 Agent 的授权**

不论是企业内部研发的一方工具，还是第三方商业化工具、开源工具，都不能默认信任 Agent。工具端必须承担认证和授权职责，同样需要判断：当前这个 Agent，和这个 Agent 所代表的用户，是否被允许访问该工具及其对应数据。

3. **Agent 出方向访问能力的约束**

为了配合外部工具或服务完成授权，Agent 本身需要具备标准化的受控访问能力，例如 OAuth 客户端能力，包括 2LO 和 3LO 两种模式。如果访问的是云资源，则需要具备获取和使用短期凭证的能力，例如 IAM Role 或 STS，而不是长期持有固定密钥。

二、Agent 的专用沙箱和受控执行环境 （Agent Sandbox）

在 Agent 的执行范式下，安全风险的重心正在发生明显变化。

过去我们更关注的是输出内容本身是否安全，例如：

- 回答是否有害
- 是否泄露敏感信息
- 是否包含不当内容

但在 Agent 时代，仅仅关注“说了什么”已经不够，因为 Agent 开始真正“做事情”了。

- 它会访问网页
- 它会执行脚本
- 它会下载和上传文件
- 它会操作浏览器
- 它会调用系统接口

也正因为如此，**安全问题开始从 “输出内容安全” 转向 “执行结果是否可接受、是否可控、是否可追责 ”**。

所以Agent 一旦进入执行态，就必须被当作一个高动态、高权限敏感、高不确定性的自动化工作负载来管理。所有会造成高危结果，线上生产变更的行为都必须由人来卡控。对这样的东西，专用沙箱不是加分项，而是基本盘。

至少应该做到：

- 将 Agent 的执行环境与主业务系统隔离
- 将不同用户、不同任务、不同会话的执行过程隔离
- 限制 Agent 的文件系统、网络、浏览器和命令执行能力
- 在任务结束后自动清理临时文件、凭证和会话状态

- 在发生异常时可以快速终止执行并保留审计证据

因为一旦没有这层隔离，模型层的误判就很容易直接变成执行层事故。

三、 Agent 执行全流程的可发现、可观测、可管控、可审计

无论是身份验证还是沙箱隔离，最终都会指向一个更底层的问题：企业是否真的知道哪里有 Agent、这些 Agent 在做什么、它们连了什么、能做什么，以及企业是否真的有能力在关键时刻把它控制住，并在事后完整追溯每一环。

首先是 **可发现**。企业必须先知道系统里到底有哪些 Agent 资产存在，它们分别服务于什么业务场景，接入了哪些工具、知识库和外部系统，又具备怎样的执行能力和权限边界。

- 发现有哪些 Agent 正在运行
- 发现它们服务于哪些业务流程
- 发现它们接入了哪些工具、数据源和外部系统
- 发现它们具备哪些执行能力和权限范围
- 发现新接入的 Agent 或能力是否已经超出原有治理范围

其次是 **可观测**。企业必须知道：

- Agent 当前接收了什么输入
- 当前使用了哪些上下文
- 进行了怎样的任务拆解
- 调用了哪些工具
- 最终输出了什么动作和结果

再次是 **可管控**。企业必须能够：

- 决定哪些工具可以被调用
- 决定哪些内容可以进入主推理链
- 决定哪些高风险动作需要人工确认
- 在异常情况下中断、降级或接管 Agent

最后是 **可审计**。企业必须能够在事后回答：

- 谁发起了这个任务
- Agent 当时代表的是谁
- 它为什么调用了这个工具
- 哪个上下文影响了它的决策
- 问题最终发生在哪一个环节

如果缺少这四项能力，Agent 系统即使在功能上能够跑起来，在安全上也仍然是一个黑盒。它可能看起来很聪明，但一旦出错，企业既无法及时干预，也无法在事后完成责任界定和经验回收。

因此，站在生产环境的角度看，Agent 最先要重建的不是某一条具体规则，而是一整套围绕身份、执行、工具和审计建立起来的基础安全能力。

六、AI Agent 安全新范式下的前瞻思考

前文探讨的诸多安全设计，更多是聚焦于为 AI Agent 构建运行前的“默认安全”环境。其实在aws的博客中提到的基于Agentic AI的特性总结的安全威胁中有个很有意思的2个点：

- Overwhelming Human in the Loop：这种威胁针对的是具有人类监督和决策验证的系统，旨在利用人类的

认知局限性或破坏交互框架。

- Human Manipulation：在Agentic AI与人类用户直接交互的场景中，信任关系会降低用户的怀疑程度，增强对Agent响应和自主性的依赖。这种隐性信任和人机直接交互会带来风险，因为攻击者可以胁迫Agent操纵用户、传播虚假信息并采取隐蔽行动。

这两点深刻揭示了一个反直觉的现象：在复杂的 Agent 系统中，**过度依赖人类监督反而可能成为安全链条中最脆弱的一环**。

但其实这也是很现实的一点，类比于现在AI Coding的发展中，代码生成的效率瓶颈早已不再是 AI 的算力或吞吐量，而是人类开发者根本无法跟上 Review 的节奏来验证代码是否符合预期。当 AI 生成海量代码时，必然会逐渐演变为“用 AI 来 Review AI 生成的代码”。在这个过程中，人类最终退居二线，仅负责审计高价值的简短摘要，而大量底层的实现细节则被隐匿。

我对这种趋势其实并不持负面态度。相反，在 AI 技术高速迭代的今天，我认为这必然会成为主流范式。人类对 AI 的信任度将会不断攀升，最终人类的角色将全面转向**审计高价值、高信息密度的数据**。

然而，正是在这种高度信任、高度依赖的背景下，我们愈发的要对这类新型威胁建立更加严密的防范机制，需要在 HITL（Human-in-the-Loop） 的决策中为人类提供足够丰富且精准的上下文信息以保护决策质量，并从机制上预防“决策疲劳”漏洞，防止攻击者通过欺骗性的 AI 行为使人类超负荷运转、操纵 AI 意图或绕过现有的安全机制。

此外，由于 LLM本质上仍是一个黑盒的程序，安全防护的重心必须向 **Agent 运行时的动态表现、决策过程和最终结果**转移。综合上述思考可以发现，除了基础设施的安全加固，Agent 安全的核心其实在于运行时上下文 。上下文直接决定了 Agent 后续的决策与行为。只有让安全机制深度介入运行时上下文，才能准确分辨 Agent 的动作是否偏离了全局任务目标和预设职责。

而在传统应用安全领域，**RASP（运行时应用自我保护）** 技术被广泛用于应用运行时的实时防护。那么，在 Agent 安全体系中，我们完全可以借鉴 RASP 的核心理念，构建针对 AI Agent 的“运行时实时检测与防护”机制（即 **Agent-RASP**）。这种机制不仅能够完成实时监测，甚至可以深度介入 Agent 的运行状态，进行实时的行为检测，纠偏和高危恶意指令的即时阻断等等操作。在最近的RSAC中已经出现了类似思路产品的创业公司（Geordie AI）感兴趣的可以去看一下他们的路演

我相信，**基于运行时的动态上下文安全防护，一定会成为新时代 AI Agent 安全防御矩阵中重要的一环。**